

ARCADEPORTAL

Portal z grami przeglądarkowymi

Dokumentacja projektowa

Projektowanie i programowanie systemów internetowych I
Semestr letni 2024/2025

Przemysław Pyrzyński · Michał Napora · Maksymilian Wojtkowski

Spis treści

Spis treści	2
1. Opis funkcjonalny systemu	4
1.1. Moduł gier	4
1.2. System kont użytkowników	4
1.3. Ranking i osiągnięcia	4
1.4. Forum	4
1.5. Panel administratora	4
2. Opis technologiczny	6
2.1. Stos technologiczny	6
2.2. Architektura aplikacji	6
3. Schemat bazy danych (ERD)	7
3.1. Tabele własne	7
Users (rozszerzenie IdentityUser)	7
Games	7
GameSessions	7
Achievements	8
UserAchievements	8
Threads	8
Posts	9
3.2. Relacje między tabelami	9
4. Wdrożone zagadnienia kwalifikacyjne	10
5. Instrukcja uruchomienia	11
5.1. Uruchomienie lokalne	11
5.2. Uruchomienie przez Docker	11
6. Wnioski projektowe	12
6.1. Osiągnięte cele	12
6.2. Napotkane trudności	12
6.3. Możliwe rozszerzenia	12
Opis zagadnień kwalifikacyjnych	13
Opis szczegółowy każdego zagadnienia	14
1. Framework MVC	14
2. Framework CSS	14
3. Baza danych	14
4. Cache	14
5. Dependency manager	14
6. HTML	15
7. CSS	15
8. JavaScript	15
9. Routing	15

10. ORM	15
11. Uwierzytelnianie	15
12. Lokalizacja	15
13. Mailing	15
14. Formularze	16
15. Asynchroniczne interakcje	16
16. Konsumpcja API	16
17. Publikacja API	16
18. RWD	16
19. Logger	16
20. Deployment	16

1. Opis funkcjonalny systemu

ArcadePortal to portal z grami przeglądarkowymi umożliwiający użytkownikom rejestrację kont, rywalizację w grach zaimplementowanych w JavaScript oraz śledzenie wyników na globalnym rankingu.

1.1. Moduł gier

System obsługuje następujące gry przeglądarkowe:

- Snake — klasyczna gra węża na siatce 24×24, rysowana na elemencie canvas
- Tetris — zaawansowany Tetris z grawitacją, przyspieszeniem i podglądem następnego klocka
- Flappy Bird — gra zręcznościowa z fizyką grawitacji i generowaniem rur
- Pac-Man — klasyczny Pac-Man z duchami, power pelletami i systemem poziomów
- Blackjack — karciana gra blackjack z pełną mechaniką asów i krupiera

1.2. System kont użytkowników

- Rejestracja z walidacją danych i emailiem powitalnym
- Logowanie przez nazwę użytkownika lub adres email
- Profil użytkownika z historią rozgrywek i statystykami
- Zmiana nazwy użytkownika, avatara i hasła
- Reset hasła przez email z tokenem jednorazowym
- System ról: Admin i User

1.3. Ranking i osiągnięcia

- Globalny ranking oparty na sumie najlepszych wyników per gra
- Top 10 per gra z cachowaniem wyników (60 sekund)
- System odznak przyznawanych automatycznie po spełnieniu warunków
- Powiadomienia toast o nowych odznakach po zakończeniu gry

1.4. Forum

- Wątki ogólne i przypisane do konkretnej gry
- Tworzenie wątków, dodawanie odpowiedzi, edycja i usuwanie własnych postów
- Niezalogowani mogą czytać, zalogowani mogą pisać
- Moderacja przez administratorów

1.5. Panel administratora

- Zarządzanie użytkownikami: zmiana ról, blokowanie kont
- Zarządzanie grami: dodawanie, edycja, włączanie/wyłączanie

- Zarządzanie odznakami: tworzenie warunków per gra
- Usuwanie postów innych użytkowników
- Statystyki globalne: liczba użytkowników, sesji, wątków, postów

2. Opis technologiczny

2.1. Stos technologiczny

Warstwa	Technologia	Opis
Backend	ASP.NET Core 8 MVC	Framework MVC — kontrolery, routing, autoryzacja
ORM	Entity Framework Core 8	Mapowanie obiektowo-relacyjne, migracje, LINQ
Baza danych	SQLite	Lekka baza plikowa, idealna do projektu zaliczeniowego
Identity	ASP.NET Core Identity	System ról, hashowanie haseł, lockout, tokeny
Cache	IMemoryCache	Cachowanie rankingów w pamięci serwera
Frontend	HTML5 + CSS3	Razor Views, responsywny layout (RWD)
CSS Framework	Bootstrap 5	Grid system, komponenty UI
JavaScript	Vanilla JS (ES6+)	Logika gier, fetch API, animacje
Gry	HTML5 Canvas API	Renderowanie gier: Snake, Tetris, Flappy, Pac-Man
Mailing	MailKit + SMTP	Wysyłka maili: rejestracja, reset hasła, odznaki
Lokalizacja	IStringLocalizer / IViewLocalizer	Obsługa języków PL i EN
Konteneryzacja	Docker + Docker Compose	Pakowanie aplikacji z Mailpitem

2.2. Architektura aplikacji

Aplikacja zbudowana jest w architekturze MVC (Model-View-Controller) z dodatkową warstwą serwisów:

- Models — encje EF Core mapowane na tabele bazy danych
- DTOs — obiekty transferu danych między kontrolerami a widokami
- Controllers — obsługa requestów HTTP, walidacja, logika routingu
- Services — logika biznesowa (AchievementService, MailKitEmailService)
- Views — szablony Razor (.cshtml) z kodem HTML/CSS
- Data — ApplicationDbContext, DbSeeder, migracje EF Core

3. Schemat bazy danych (ERD)

Baza danych oparta na SQLite zawiera 7 tabel własnych oraz tabele zarządzane przez ASP.NET Core Identity (AspNetUsers, AspNetRoles, AspNetUserRoles i inne).

3.1. Tabele własne

Users (rozszerzenie IdentityUser)

Users — AspNetUsers		
Kolumna	Typ	Opis
Id PK	<code>string</code>	Klucz główny (GUID generowany przez Identity)
UserName	<code>string</code>	Unikalna nazwa użytkownika
Email	<code>string</code>	Adres email użytkownika
PasswordHash	<code>string</code>	Hash hasła (PBKDF2, zarządzany przez Identity)
AvatarUrl	<code>string?</code>	Opcjonalny URL avatara użytkownika
CreatedAt	<code>DateTime</code>	Data i czas rejestracji (UTC)

Games

Games		
Kolumna	Typ	Opis
Id PK	<code>int</code>	Klucz główny
Slug	<code>string</code>	Unikalny identyfikator tekstowy (np. "snake")
Title	<code>string</code>	Tytuł gry wyświetlany użytkownikom
Description	<code>string?</code>	Opcjonalny opis gry
ThumbnailUrl	<code>string?</code>	URL miniaturki gry
IsActive	<code>bool</code>	Czy gra jest widoczna dla graczy
CreatedAt	<code>DateTime</code>	Data dodania gry (UTC)

GameSessions

GameSessions		
Kolumna	Typ	Opis
Id PK	<code>int</code>	Klucz główny
UserId FK	<code>string</code>	Referencja do Users

GameId FK	int	Referencja do Games
Score	int	Wynik zdobyty w tej sesji
DurationSeconds	int	Czas trwania rozgrywki w sekundach
PlayedAt	DateTime	Data i czas rozgrywki (UTC)

Achievements

Achievements		
Kolumna	Typ	Opis
Id PK	int	Klucz główny
GameId FK	int	Referencja do Games (odznaka należy do gry)
Name	string	Nazwa odznaki wyświetlana użytkownikowi
Description	string	Opis warunku zdobycia odznaki
IconUrl	string?	Emoji lub URL ikony odznaki
Condition	string	Warunek w formacie "score>=100" lub "duration>=60"

UserAchievements

UserAchievements		
Kolumna	Typ	Opis
Id PK	int	Klucz główny
UserId FK	string	Referencja do Users
AchievementId FK	int	Referencja do Achievements
UnlockedAt	DateTime	Data i czas odblokowania odznaki (UTC)

Threads

Threads		
Kolumna	Typ	Opis
Id PK	int	Klucz główny
Title	string	Tytuł wątku (max 200 znaków)
UserId FK	string	Autor wątku — referencja do Users
GameId FK	int?	Opcjonalna referencja do Games (null = wątek ogólny)
IsPinned	bool	Czy wątek jest przypięty przez moderatora
CreatedAt	DateTime	Data i czas utworzenia wątku (UTC)

Posts

Posts		
Kolumna	Typ	Opis
Id PK	int	Klucz główny
ThreadId FK	int	Referencja do Threads
UserId FK	string	Autor posta — referencja do Users
Body	string	Treść posta (max 5000 znaków)
CreatedAt	DateTime	Data i czas utworzenia posta (UTC)
UpdatedAt	DateTime?	Data ostatniej edycji (null = nie edytowano)

3.2. Relacje między tabelami

Tabela A	Relacja	Tabela B	Opis
Users	1 : N	GameSessions	Jeden użytkownik może mieć wiele sesji gry
Games	1 : N	GameSessions	Jedna gra może mieć wiele sesji
Users	1 : N	UserAchievements	Jeden użytkownik może mieć wiele odznak
Achievements	1 : N	UserAchievements	Jedna odznaka może być odblokowana przez wielu
Games	1 : N	Achievements	Jedna gra definiuje wiele odznak
Users	1 : N	Threads	Jeden użytkownik może tworzyć wiele wątków
Games	0..1 : N	Threads	Wątek opcjonalnie przypisany do gry
Threads	1 : N	Posts	Jeden wątek zawiera wiele postów
Users	1 : N	Posts	Jeden użytkownik może pisać wiele postów

4. Wdrożone zagadnienia kwalifikacyjne

#	Zagadnienie	Implementacja
1	Framework MVC	ASP.NET Core 8 MVC — kontrolery, modele, widoki Razor
2	Framework CSS	Bootstrap 5 — grid, komponenty; własny system CSS z variables
3	Baza danych	SQLite przez Entity Framework Core 8
4	Cache	IMemoryCache — cachowanie rankingów i leaderboardów
5	Dependency manager	NuGet — zarządzanie pakietami .NET
6	HTML	Razor Views (.cshtml) — szkielet aplikacji
7	CSS	Własne pliki CSS per moduł (auth, forum, game, admin...)
8	JavaScript	Logika gier w Vanilla JS, sterowanie, pętla gry
9	Routing	Atrybutowy i konwencjonalny routing MVC, pretty URLs
10	ORM	Entity Framework Core — DbContext, migracje, LINQ
11	Uwierzytelnianie	ASP.NET Core Identity — role, hasła, logout, tokeny
12	Lokalizacja	IStringLocalizer / IViewLocalizer, pliki .resx, PL/EN
13	Mailing	MailKit + SMTP — rejestracja, reset hasła, odznaki
14	Formularze	Razor Forms z walidacją, AntiForgeryToken
15	Asynchroniczne interakcje	Fetch API — zapis wyników, asynchroniczne zapytania
16	Konsumpcja API	Własne REST API /api/scores konsumowane przez JS gier
17	Publikacja API	Endpoint /api/scores (POST) z autoryzacją i CSRF
18	RWD	Bootstrap Grid + media queries, responsywny layout
19	Logger	ILogger<T> we wszystkich kontrolerach i serwisach
20	Deployment	Docker + Docker Compose z Mailpit i volume na bazę

5. Instrukcja uruchomienia

5.1. Uruchomienie lokalne

Wymagania: .NET 8 SDK, Mailpit

- Sklonuj repozytorium: `git clone <url>`
- Zainstaluj Mailpit i uruchom: `mailpit.exe`
- W katalogu projektu: `dotnet ef database update`
- Uruchom aplikację: `dotnet run`
- Otwórz przeglądarkę: `http://localhost:5000`
- Skrzynka mailowa: `http://localhost:8025`

5.2. Uruchomienie przez Docker

Wymagania: Docker Desktop

- Sklonuj repozytorium i przejdź do katalogu projektu
- Uruchom: `docker compose up --build`
- Aplikacja dostępna pod: `http://localhost:8080`
- Skrzynka mailowa: `http://localhost:8025`

Przy pierwszym uruchomieniu seeder automatycznie tworzy role, konto admina, gry z odznakami oraz 10 testowych użytkowników z losowymi wynikami.

6. Wnioski projektowe

6.1. Osiągnięte cele

- Zrealizowano portal z 5 grami przeglądarkowymi w pełni zintegrowanymi z systemem zapisywania wyników
- Zaimplementowano kompletny system kont z rolami, uwierzytelnianiem i resetem hasła przez email
- Stworzono forum dyskusyjne z moderacją i wątkami przypisanymi do gier
- Skonteneryzowano aplikację przez Docker z pełną izolacją środowiska

6.2. Napotkane trudności

- Integracja własnego systemu menu gier z jednym widokiem Play.cshtml wymagała przemyślenia architektury JS
- Konfiguracja ASP.NET Core Identity z własnym modelem User wymagała znajomości cyklu życia tokenów i claims

6.3. Możliwe rozszerzenia

- Dodanie OAuth2 (logowanie przez Google/GitHub)
- System turniejów i rozgrywek wieloosobowych
- Migracja bazy danych z SQLite na PostgreSQL dla środowiska produkcyjnego

Opis zagadnień kwalifikacyjnych

W tej sekcji opisujemy szczegółowo każde z 20 zagadnień kwalifikacyjnych — gdzie jest zaimplementowane w projekcie oraz jak można sprawdzić że działa.

#	Zagadnienie	Gdzie w projekcie	Jak sprawdzić
1	Framework MVC	<code>Controllers/*.cs</code> , <code>Program.cs</code> , <code>Views/*.cshtml</code>	Wejść na <code>/Home</code> , <code>/Game</code> , <code>/Forum</code> — każda strona to inny kontroler
2	Framework CSS	<code>Views/Shared/_Layout.cshtml</code> (Bootstrap CDN), <code>wwwroot/css/*.css</code>	Otwórz DevTools → Elements → sprawdź klasy Bootstrap i własne
3	Baza danych	<code>Data/AppDbContext.cs</code> , <code>appsettings.json</code> , <code>arcade.db</code>	Otwórz plik <code>arcade.db</code> w DB Viewerze w Visual Studio
4	Cache	<code>Controllers/GameController.cs</code> (<code>GetOrCreateAsync</code>), <code>Controllers/LeaderboardController.cs</code>	Zagraj grę, sprawdź wynik — przez 60s top 10 się nie zmienia
5	Dependency manager	<code>ArcadeProject.csproj</code> (<code><PackageReference></code>), <code>Dockerfile</code>	Otwórz <code>.csproj</code> — widać wszystkie paczki NuGet
6	HTML	<code>Views/**/*.cshtml</code> — każdy widok to szablon HTML	Ctrl+U na dowolnej stronie — widać wygenerowany HTML
7	CSS	<code>wwwroot/css/gamePlay.css</code> , <code>auth.css</code> , <code>forum.css</code> , <code>admin.css</code> , <code>leaderboard.css</code> , <code>profile.css</code>	DevTools → Network → CSS — każda strona ładuje swój plik CSS
8	JavaScript	<code>wwwroot/js/gamePlay.js</code> , <code>wwwroot/js/games/snake.js</code> , <code>tetris.js</code> , <code>flappy.js</code> , <code>pacman.js</code> , <code>blackjack.js</code>	DevTools → Sources → <code>wwwroot/js/</code>
9	Routing	<code>Program.cs</code> (<code>MapControllerRoute</code>), <code>Controllers/GameController.cs</code> (<code>[HttpPost("/api/scores")]</code>)	Przejdź na <code>/Game/Play/snake</code> — "snake" to parametr z URL
10	ORM	<code>Data/AppDbContext.cs</code> , <code>Models/*.cs</code> , <code>Migrations/</code>	Folder <code>Migrations/</code> — każda migracja to zmiana w bazie
11	Uwierzytelnianie	<code>Controllers/AccountController.cs</code> , <code>Program.cs</code> (<code>AddIdentity</code>), <code>Data/DbSeeder.cs</code>	Wejść na <code>/Admin</code> bez logowania — przekieruje na <code>/Account/Login</code>
12	Lokalizacja	<code>Controllers/CultureController.cs</code> , <code>Resources/**/*.resx</code>	Kliknij EN w navbarze — strona główna zmienia się na angielski
13	Mailing	<code>Services/MailKitEmailService.cs</code> , <code>Services/IEmailService.cs</code>	Zarejestruj konto → sprawdź <code>http://localhost:8025</code>
14	Formularze	<code>Views/Account/Register.cshtml</code> , <code>Views/Forum/Create.cshtml</code> , <code>DTOs/AccountDtos.cs</code>	Spróbuj zarejestrować się z pustymi polami — pojawią się błędy
15	Async interakcje	<code>wwwroot/js/gamePlay.js</code> (funkcja <code>saveScore()</code> z <code>fetch()</code>)	DevTools → Network → zagraj grę → widać POST <code>/api/scores</code>
16	Konsumpcja API	<code>wwwroot/js/gamePlay.js</code> (<code>fetch POST /api/scores</code>)	Jak wyżej — JS konsumuje endpoint wystawiony przez kontroler

17	Publikacja API	<code>Controllers/GameController.cs</code> (metoda <code>SaveScore</code> , <code>[HttpPost("/api/scores")]</code>)	DevTools → kliknij request /api/scores → Preview → JSON
18	RWD	<code>wwwroot/css/gamePlay.css</code> (@media max-width: 900px), <code>_Layout.cshtml</code> (<code>navbar-expand-md</code>)	DevTools → Ctrl+Shift+I → symuluj iPhone → sprawdź układ
19	Logger	<code>Controllers/HomeController.cs</code> , <code>GameController.cs</code> , <code>AccountController.cs</code> , <code>Services/MailKitEmailService.cs</code>	<code>docker compose logs app -f</code> → zaloguj się → widać logi
20	Deployment	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>.dockerignore</code>	Na innym komputerze: <code>docker</code> <code>compose up --build</code> → działa bez .NET

Opis szczegółowy każdego zagadnienia

1. Framework MVC

Projekt używa ASP.NET Core 8 MVC. Każda strona to kontroler który obsługuje żądania i zwraca widok. Na przykład `HomeController` obsługuje stronę główną, `GameController` obsługuje listę gier i uruchamianie gry, `ForumController` obsługuje forum itd. Routing jest skonfigurowany w `Program.cs` jako.

2. Framework CSS

Używamy Bootstrap 5 który jest dołączony przez CDN w pliku `_Layout.cshtml`. Bootstrap obsługuje responsywny grid i nawigację. Oprócz Bootstrap mamy własne pliki CSS dla każdego modułu — osobny plik dla strony gry, forum, logowania, panelu admina itd. Dzięki temu style się nie mieszają i każda strona ładuje tylko to czego potrzebuje.

3. Baza danych

Używamy SQLite bo jest prosty w użyciu i nie wymaga osobnego serwera. Baza to jeden plik `arcade.db`. W Dockerze ten plik jest trzymany na osobnym volume żeby nie zginął przy rebuild.

4. Cache

`IMemoryCache` jest wstrzyknięty do `GameController` i `LeaderboardController`. Ranking top 10 per gra jest cachowany przez 60 sekund. Globalny ranking jest cachowany przez 2 minuty. Po zapisaniu nowego wyniku cache jest usuwany przez `_cache.Remove()` żeby przy następnym wejściu dane były świeże.

5. Dependency manager

NuGet jest standardowym menedżerem pakietów dla .NET. Wszystkie zależności (EF Core, Identity, MailKit, Bootstrap itp.) są zapisane w pliku `ArcadeProject.csproj` w sekcji `PackageReference`. Komenda `dotnet restore` pobiera je automatycznie. W `Dockerfile` `restore` jest jako osobna warstwa co przyspiesza buildy.

6. HTML

Używamy Razor Views — pliki .cshtml które są szablonami HTML z możliwością wstawiania kodu C# przez składnię @. Każdy kontroler ma swój folder w Views/. Plik _Layout.cshtml jest wspólnym szkieletem dla wszystkich stron — zawiera navbar, footer i miejsca na sekcje Scripts i Styles.

7. CSS

Oprócz Bootstrap mamy własne pliki CSS. Globalny styl jest w site.css. Każda sekcja ma swój plik — auth.css dla logowania i rejestracji, forum.css itd. Używamy CSS Custom Properties (zmienne) dla kolorów motywu.

8. JavaScript

Logika gier jest napisana w czystym JavaScript (ES6+) bez frameworków. Plik gamePlay.js to silnik strony gry — obsługuje menu, zapis wyników przez fetch i powiadomienia o odznakach. Każda gra ma swój plik (snake.js, tetris.js itd.) który implementuje funkcję window.gameStart() wywoływaną przez gamePlay.js po kliknięciu START. Komunikacja z serwerem przez Fetch API.

9. Routing

W Program.cs mamy standardowy routing MVC. Dzięki temu URL /Game/Play/snake przekazuje "snake" jako parametr slug do metody Play() w GameController.

10. ORM

Entity Framework Core 8 jest naszym ORM. ApplicationDbContext dziedziczy po IdentityDbContext i definiuje DbSety dla wszystkich encji. Relacje (np. kaskadowe usuwanie) są skonfigurowane w metodzie OnModelCreating. Zapytania piszemy w LINQ. Migracje są w folderze Migrations/.

11. Uwierzytelnianie

ASP.NET Core Identity zarządza użytkownikami i rolami. Mamy dwie role: Admin i User tworzone przez seeder przy starcie. Hasła są hashowane algorytmem PBKDF2. Po 5 nieudanych próbach konto jest blokowane na 5 minut. Atrybut [Authorize(Roles="Admin")] na AdminController sprawia że tylko admini mają dostęp do panelu. Logowanie działa przez username lub email.

12. Lokalizacja

Obsługujemy dwa języki: polski i angielski. W navbarze jest przycisk PL/EN który wysyła POST do CultureController i zapisuje wybrany język w ciasteczku na rok. Tłumaczenia są w plikach .resx w folderze Resources/ — osobny plik per kontroler i widok. W widokach używamy @inject IViewLocalizer Loc i @Loc["klucz"] zamiast hardcodowanych tekstów.

13. Mailing

Do wysyłania emaili używamy biblioteki MailKit. Stworzyliśmy interfejs IEmailService z metodami SendWelcomeAsync, SendPasswordResetAsync i SendAchievementAsync. Implementacja MailKitEmailService łączy się przez SMTP do Mailpit (lokalnie i w Dockerze) lub zewnętrznego serwera. Jeśli wysyłka się nie uda to błąd jest logowany ale nie przerywa działania aplikacji (try/catch).

14. Formularze

Formularze w aplikacji używają Razor Tag Helpers — `asp-for`, `asp-action`, `asp-validation-for`. Walidacja jest definiowana przez atrybuty w klasach DTO np. `[Required]`, `[MinLength(6)]`, `[EmailAddress]`. Każdy formularz POST jest zabezpieczony przez `[ValidateAntiForgeryToken]` który chroni przed atakami CSRF. Błędy walidacji wracają do widoku przez `ModelState`.

15. Asynchroniczne interakcje

Po zakończeniu gry JavaScript wywołuje `fetch()` POST `/api/scores` bez przeładowania strony. Wysyła JSON z wynikiem i czasem gry. Po odebraniu odpowiedzi aktualizuje pasek statusu (wynik, rekord osobisty) i pokazuje powiadomienia o odznakach jako animowane elementy HTML dodawane do body. Wszystko dzieje się bez reload.

16. Konsumpcja API

Plik `gamePlay.js` konsumuje endpoint `/api/scores` przez natywne `fetch()` API przeglądarki. Wysyła JSON body z `gameSlug`, `score` i `durationSeconds`. Dołącza też token CSRF z ukrytego pola formularza w nagłówku `RequestVerificationToken`. Obsługuje błędy — jeśli serwer zwróci 401 (niezalogowany) wyświetla komunikat bez crashowania gry.

17. Publikacja API

Metoda `SaveScore()` w `GameController` jest wystawiona jako REST endpoint. Ma atrybut `[HttpPost("/api/scores")]` i `[ValidateAntiForgeryToken]`. Sprawdza czy użytkownik jest zalogowany (401 jeśli nie), czy gra istnieje (404 jeśli nie). Zapisuje sesję do bazy, wywołuje `AchievementService` i zwraca JSON: `{ message, score, newAchievements: [...] }`.

18. RWD

Strona jest responsywna dzięki Bootstrap Grid i własnym media queries. Strona gry używa CSS Grid który automatycznie przełącza się z 2 kolumn (gra + sidebar) na 1 kolumnę poniżej 900px. Nawigacja korzysta z `navbar-expand-md` Bootstrap — menu zwija się do hamburgera na telefonach. Siatka kart gier używa `auto-fill` z `minmax(260px, 1fr)` co automatycznie dobiera liczbę kolumn.

19. Logger

Kontroler i serwis ma wstrzyknięty `ILogger<T>` przez konstruktor. Logujemy informacje przy kluczowych akcjach: załadowanie strony głównej, logowanie, rejestracja, wejście w grę, zapis wyniku, przyznanie odznaki. Błędy wysyłki emaila logujemy przez `LogError`. W Dockerze logi widać przez komendę `docker compose logs app -f`.

20. Deployment

Aplikacja jest skonteneryzowana w Dockerze. Dockerfile używa multi-stage build — najpierw obraz z SDK do kompilacji, potem lżejszy obraz `aspnet` do uruchomienia. `docker-compose.yml` uruchamia dwa kontenery: aplikację na porcie 8080 i Mailpit na portach 8025/1025. Baza SQLite jest na volume `db-data` który przeżywa rebuildy. Na innym komputerze wystarczy Docker Desktop i komenda `docker compose up --build`.